

HuggingFace를 활용한
인공지능 웹 애플리케이션 개발

Chapter 4

이미지 생성 서비스 만들기

Stable Diffusion · ControlNet · LoRA · FastAPI · 모델 최적화

2025년판 개정증보

목 차

4장 서문	3
4.1 Stable Diffusion 실습 개요	4
(1) Stable Diffusion의 작동 원리	4
(2) diffusers 라이브러리 소개 및 설치	7
(3) 모델 로딩 및 Hugging Face 토큰 발급	9
(4) CPU vs GPU 성능 비교	13
(5) 첫 이미지 생성 실습: 텍스트 → PNG 파일 저장	15
4.2 프롬프트 입력 → 이미지 생성 API	18
(1) FastAPI로 이미지 생성 요청 API 구현	18
(2) 이미지 파일 또는 base64 인코딩 반환	21
(3) Swagger UI 테스트 및 Streamlit 앱 연동	24
(4) torch.autocast / fp16 최적화	27
4.3 ControlNet과 LoRA로 이미지 변환	29
(1) ControlNet 개념 및 조건 입력 방식	29
(2) ControlNet Canny 예시 실습	32
(3) LoRA 개념 및 적용 방법	35
(4)(5) 이미지 업로드 → 변환 반환 API 구현	37
4.4 서버에서 모델 최적화하기	41
4장 마무리 및 연습 문제	47

4장 이미지 생성 서비스 만들기

이 장에서는 Hugging Face의 `diffusers` 라이브러리를 활용하여 Stable Diffusion 기반의 텍스트-투-이미지(Text-to-Image) 생성 모델을 웹 API로 구현하는 전 과정을 다룹니다. 텍스트 프롬프트 한 줄만으로 고품질 이미지를 생성하고, 이를 실제 서비스 수준의 REST API로 배포하는 방법을 단계별로 학습합니다.

Stable Diffusion은 2022년 CompVis 연구팀이 공개한 이래 AI 이미지 생성 분야를 완전히 바꾸어 놓았습니다. 오픈소스 기반으로 누구나 사용할 수 있으며, Hugging Face의 `diffusers` 라이브러리를 통해 단 몇 줄의 코드로 강력한 이미지 생성 기능을 구현할 수 있습니다.

4.1절에서는 Stable Diffusion의 작동 원리와 `diffusers` 설치 및 첫 이미지 생성 실습을 진행합니다.

4.2절에서는 FastAPI와 결합하여 프로덕션 수준의 이미지 생성 API를 구축합니다. 4.3절에서는 ControlNet과 LoRA를 활용한 이미지 변환 서비스를 구현하고, 4.4절에서는 GPU 메모리 최적화와 응답 속도 향상 기법을 집중적으로 다룹니다.

이 장에서 배울 내용

- Stable Diffusion의 UNet, VAE, CLIP 구조와 확산 과정 이해
- `diffusers` 라이브러리로 텍스트 → 이미지 생성 실습
- FastAPI로 이미지 생성 REST API 서버 구축 (파일 반환 / base64 반환)
- ControlNet(Canny, Pose, Depth)으로 구조 보존 이미지 변환
- LoRA 가중치 로드 및 스타일 전환 실습
- fp16, CPU Offload, xFormers 등 GPU 메모리 최적화 기법

실습 환경 요구 사항

- Python 3.9 이상
- `pip install diffusers transformers accelerate torch torchvision`
- `pip install fastapi uvicorn python-multipart Pillow opencv-python`
- GPU (VRAM 6GB 이상 권장) – CPU 실행은 매우 느림 (5~30분/장)
- Hugging Face 계정 및 액세스 토큰 (일부 모델 다운로드 시 필요)
- Google Colab Pro (A100/T4 무료 사용) 으로도 실행 가능

4.1 Stable Diffusion 실습 개요

(1) Stable Diffusion의 작동 원리 요약 (텍스트 → 잠재공간 → 이미지)

Stable Diffusion은 세 가지 핵심 신경망 구성 요소가 상호 협력하여 텍스트에서 이미지를 생성하는 잠재 확산 모델(Latent Diffusion Model, LDM)입니다. 픽셀 공간에서 직접 확산(diffusion)을 수행하는 기존 방식과 달리, 압축된 잠재 공간(latent space)에서 확산을 수행하기 때문에 계산 효율이 훨씬 뛰어납니다.

전체 생성 과정은 크게 세 단계로 나눌 수 있습니다.

구성 요소	역할	세부 설명
CLIP Text Encoder	텍스트 → 임베딩 벡터	입력 프롬프트를 토큰화하고 768/1024차원의 의미 벡터로 변환. 이 벡터가 UNet의 Cross-Attention을 통해 이미지 생성을 유도함
UNet (Denoising Network)	노이즈 예측 및 제거	잠재 벡터에 추가된 가우시안 노이즈를 단계적으로 제거하는 역할. 텍스트 임베딩과 Cross-Attention으로 결합되어 텍스트 조건에 맞는 이미지를 생성
VAE (Variational AutoEncoder)	잠재공간 ↔ 픽셀공간 변환	Encoder: 실제 이미지 → 잠재 벡터 (4× 64× 64). Decoder: 최종 잠재 벡터 → 실제 이미지 픽셀 (3× 512× 512)
Scheduler (DDPM/DDIM 등)	확산 스텝 제어	몇 번의 역확산 스텝으로 노이즈를 제거할지 결정. DDIM은 20~50 스텝으로 고품질 이미지 생성 가능

생성 흐름을 순서대로 따라가면 다음과 같습니다.

🔗 Stable Diffusion 이미지 생성 단계별 흐름

① 텍스트 인코딩: 프롬프트 문자열 → CLIP Text Encoder → 텍스트 임베딩 벡터 (shape: [1, 77, 768])

- ② 초기 노이즈 생성: 랜덤 가우시안 노이즈로 잠재 벡터 초기화 (shape: [1, 4, 64, 64])
- ③ 역확산 루프 (Denoising Loop, 기본 50 스텝):
- UNet이 현재 잠재 벡터의 노이즈 성분 예측
 - Scheduler가 예측된 노이즈를 바탕으로 잠재 벡터 업데이트
 - 텍스트 임베딩이 Cross-Attention을 통해 각 스텝을 안내
- ④ VAE 디코딩: 최종 정제된 잠재 벡터 → VAE Decoder → 512× 512 픽셀 이미지 (RGB)
- ⑤ 후처리: [0, 1] 범위 정규화 → PIL.Image 변환 → PNG/JPEG 저장

Classifier-Free Guidance(CFG)는 이미지 품질과 프롬프트 충실도를 조절하는 핵심 파라미터입니다. CFG 스케일(guidance_scale)이 높을수록 프롬프트와 더 가깝지만 다양성이 줄어들고, 낮을수록 자유롭지만 프롬프트에서 벗어날 수 있습니다.

guidance_scale 값	특성	권장 사용 상황
1.0 ~ 3.0	프롬프트 충실도 낮음, 매우 다양한 결과	창의적 실험, 예술적 자유
5.0 ~ 8.0	균형잡힌 품질, 중간 충실도	일반적 생성, 가장 많이 사용
9.0 ~ 12.0	높은 충실도, 선명하고 정확한 결과	정밀한 프롬프트 묘사가 필요할 때
13.0 이상	과도한 충실도, 색상 왜곡 가능	특수한 경우에만 사용 권장

(2) diffusers 라이브러리 소개 및 설치

diffusers는 Hugging Face가 개발한 확산 모델(Diffusion Model) 전용 라이브러리입니다. Stable Diffusion 외에도 DALL-E 2, Imagen, DeepFloyd IF 등 다양한 생성 모델을 일관된 인터페이스로 사용할 수 있습니다.

구성 모듈	설명
StableDiffusionPipeline	기본 txt2img 파이프라인. 프롬프트 → 이미지 생성
StableDiffusionImg2ImgPipeline	이미지+프롬프트로 이미지 변형 (img2img)
StableDiffusionInpaintPipeline	마스크 영역만 재생성 (인페인팅)
StableDiffusionControlNetPipeline	ControlNet 조건을 결합한 구조 보존 생성
StableDiffusionXLPipeline	SDXL: 1024× 1024 고해상도 생성 지원
AutoPipelineForText2Image	모델 구조를 자동 감지하여 적절한 파이프라인 선택
DiffusionPipeline.from_pretrained()	모든 확산 모델 공통 로드 메서드

설치는 pip 한 줄로 완료됩니다. 다만 버전 간 호환성 문제가 발생할 수 있으므로 아래의 검증된 버전 조합을 사용하길 권장합니다.

```
# —— 기본 설치
pip install diffusers==0.27.2 transformers==4.40.1 accelerate==0.29.3
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121

# —— 선택적 추가 패키지 ——
# xFormers: Attention 연산 최적화 (VRAM ~30% 절감)
pip install xformers==0.0.25

# ControlNet, img2img 실습용
pip install opencv-python-headless Pillow

# API 서버 구축용
pip install fastapi uvicorn python-multipart

# —— 설치 확인
python -c "import diffusers; print(diffusers.__version__)"
# 출력: 0.27.2
```

[코드 4-1] diffusers 및 관련 패키지 설치

△ torch와 CUDA 버전이 맞지 않으면 GPU 가속이 작동하지 않습니다. `nvidia-smi` 명령으로 현재 CUDA 버전을 확인 후, PyTorch 공식 사이트(<https://pytorch.org/get-started/locally/>)에서 해당 버전에 맞는 설치 명령을 복사하세요.

(3) 모델 로딩: CompVis/stable-diffusion-v1-4, stabilityai/stable-diffusion-2 및 Hugging Face 토큰 발급

Stable Diffusion 모델을 처음 로드할 때는 Hugging Face Hub에서 수 GB의 모델 파일을 다운로드 합니다. 일부 모델은 라이선스 동의가 필요하며, 이 경우 Hugging Face 액세스 토큰(Access Token)이 요구됩니다.

Hugging Face 액세스 토큰 발급 방법

1. <https://huggingface.co> 에 접속하여 계정을 생성하거나 로그인합니다.
2. 우측 상단 프로필 아이콘 → Settings → Access Tokens 메뉴로 이동합니다.
3. [New token] 버튼을 클릭하고 토큰 이름을 입력합니다. Role은 read로 설정합니다.
4. 생성된 토큰(hf_로 시작하는 문자열)을 복사하여 안전한 곳에 저장합니다.
5. 모델 페이지(예: CompVis/stable-diffusion-v1-4)에 접속하여 사용 약관(License Agreement)에 동의합니다.

```
# — 방법 1: 커맨드라인 로그인 (권장) —
# 터미널에서 실행. 프롬프트에 발급받은 토큰을 붙여넣으면 됩니다.
huggingface-cli login

# — 방법 2: 코드 내에서 직접 로그인 —
from huggingface_hub import login

# 토큰을 환경변수로 관리하는 것이 보안상 권장됩니다
import os
hf_token = os.environ.get("HF_TOKEN", "여기에_토큰_직접_입력_비권장")
login(token=hf_token)
```



```
# — 방법 3: 환경변수로 자동 인증 —
# .env 파일 또는 OS 환경변수에 설정
# export HUGGING_FACE_HUB_TOKEN=hf_XXXXXXXXXXXXXXXXX

# Python에서 읽기
from dotenv import load_dotenv
load_dotenv()
token = os.environ["HUGGING_FACE_HUB_TOKEN"]
```

[코드 4-2] Hugging Face 토큰 발급 후 로그인 방법

주요 Stable Diffusion 모델별 특징을 비교합니다.

모델 ID	버전	해상도	VRAM 요구	특징
CompVis/stable-diffusion-v1-4	SD 1.4	512× 512	4GB+	최초 공개 버전, 가장 광범인튜닝 모델 호환
runwayml/stable-diffusion-v1-5	SD 1.5	512× 512	4GB+	SD 1.4 개선판, 현재 가장용되는 기본 모델
stabilityai/stable-diffusion-2	SD 2.0	512× 512	6GB+	OpenCLIP 채용, 768 해상도 버전도 별도 제공
stabilityai/stable-diffusion-2-1	SD 2.1	768× 768	8GB+	SD 2.0 개선, 파인튜닝 친화
stabilityai/stable-diffusion-xl-base-1.0	SDXL	1024× 1024	10GB+	이중 UNet 구조, 최고 품질 GPU 필요

```
import torch
from diffusers import StableDiffusionPipeline, DiffusionPipeline

# — SD v1.5 로딩 (가장 범용적) —
model_id_v15 = "runwayml/stable-diffusion-v1-5"

pipe_v15 = StableDiffusionPipeline.from_pretrained(
    model_id_v15,
    torch_dtype=torch.float16, # fp16으로 VRAM 절감
```

```

use_safetensors=True,      # safetensors 형식 우선 로드 (보안, 속도)
safety_checker=None,      # 안전 필터 비활성 (연구/실습용)
requires_safety_checker=False
)

pipe_v15 = pipe_v15.to("cuda")

# — SD v2 로딩 —
model_id_v2 = "stabilityai/stable-diffusion-2"

pipe_v2 = DiffusionPipeline.from_pretrained(
    model_id_v2,
    torch_dtype=torch.float16,
    use_safetensors=True
).to("cuda")

# — SDXL 로딩 (고사양 GPU 필요) —
model_id_xl = "stabilityai/stable-diffusion-xl-base-1.0"

pipe_xl = DiffusionPipeline.from_pretrained(
    model_id_xl,
    torch_dtype=torch.float16,
    use_safetensors=True,
    variant="fp16"
).to("cuda")

print("모든 모델 로드 완료")

```

[코드 4-3] SD v1.5, v2, SDXL 모델 로딩 비교

★ use_safetensors=True 옵션을 반드시 사용하세요. safetensors 형식은 기존 .bin 형식보다 로드 속도가 빠르고, 악성 코드가 포함된 pickle 파일을 방지하는 보안 이점이 있습니다.

(4) CPU vs GPU 성능 비교 (Colab / 로컬)

Stable Diffusion은 GPU 없이 CPU에서도 실행할 수 있지만 속도 차이가 극단적입니다. 아래 표는 512× 512 이미지 1장을 50 스텝으로 생성할 때의 실측 시간입니다.

실행 환경	장치	정밀도	1장 생성 시간	비고
일반 PC (로컬)	Intel Core i9 CPU	fp32	약 25~ 40분	실용적 사용 불가
구글 Colab 무료	NVIDIA T4 GPU 15GB	fp16	약 15~ 25초	무료, 연결 시간 제한
구글 Colab Pro	NVIDIA A100 GPU 40GB	fp16	약 4~ 8초	고품질 실습 환경
로컬 워크스테이션	NVIDIA RTX 3060 12GB	fp16	약 10~ 18초	가성비 좋은 개인 환경
로컬 워크스테이션	NVIDIA RTX 4090 24GB	fp16	약 2~ 5초	최고 성능 개인 환경
클라우드 서버	NVIDIA A10G 24GB (AWS)	fp16	약 6~ 12초	프로덕션 배포 적합

```
# 실행 환경 자동 감지 및 최적 설정
import torch
from diffusers import StableDiffusionPipeline

def load_pipeline_for_env(model_id: str):
    """ 실행 환경에 따라 자동으로 최적 설정으로 파이프라인 로드 """
    if torch.cuda.is_available():
        # GPU 사용 가능: fp16으로 로드
        vram_gb = torch.cuda.get_device_properties(0).total_memory / 1e9
        print(f"GPU 감지: {torch.cuda.get_device_name(0)} ({vram_gb:.1f} GB VRAM)")
        pipe = StableDiffusionPipeline.from_pretrained(
            model_id,
            torch_dtype=torch.float16,
            use_safetensors=True,
            safety_checker=None,
            requires_safety_checker=False
        ).to("cuda")
```

```
if vram_gb < 8:
    # VRAM이 8GB 미만이면 추가 메모리 최적화
    pipe.enable_attention_slicing()
    print("낮은 VRAM 환경: attention slicing 활성화")
elif hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
    # Apple Silicon (M1/M2/M3)
    print("Apple Silicon MPS 감지")
    pipe = StableDiffusionPipeline.from_pretrained(
        model_id,
        torch_dtype=torch.float32, # MPS는 fp16 미지원
        use_safetensors=True,
        safety_checker=None,
        requires_safety_checker=False
    ).to("mps")
else:
    # CPU 폴백
    print("경고: GPU 없음. CPU 모드로 실행합니다 (매우 느림).")
    pipe = StableDiffusionPipeline.from_pretrained(
        model_id,
        torch_dtype=torch.float32,
        use_safetensors=True,
        safety_checker=None,
        requires_safety_checker=False
    )
return pipe

pipe = load_pipeline_for_env("runwayml/stable-diffusion-v1-5")
```

[코드 4-4] 실행 환경 자동 감지 파이프라인 로더

(5) 첫 이미지 생성 실습: 텍스트 → PNG 파일 저장

환경 설정이 완료되었으면 이제 첫 번째 이미지를 생성해 봅시다. Stable Diffusion으로 좋은 이미지를 생성하려면 프롬프트를 영어로 상세하게 작성하는 것이 중요합니다.

🔗 효과적인 프롬프트 작성 원칙

긍정 프롬프트(positive prompt): 원하는 내용을 구체적으로 서술

예) "a majestic white lion in a mystical forest, golden hour lighting, 8K, photorealistic"

부정 프롬프트(negative prompt): 원하지 않는 요소를 제거

예) "blurry, low quality, watermark, text, deformed, ugly, bad anatomy"

스타일 키워드 추가: "4K", "8K", "photorealistic", "oil painting", "anime style", "cinematic"

아티스트 참조: "in the style of Van Gogh", "Greg Rutkowski style" 등

```
import torch
from diffusers import StableDiffusionPipeline
from PIL import Image
import os
import time

# —— 파이프라인 로드 ——
pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    use_safetensors=True,
    safety_checker=None,
    requires_safety_checker=False
).to("cuda")

# —— 프롬프트 설정
```

```
prompt = (
```

```
"a majestic white lion resting in a mystical ancient forest, "  
"golden hour lighting, ultra detailed fur, 8K resolution, "  
"photorealistic, award-winning photography, cinematic composition"  
)  
negative_prompt = (  
    "blurry, low quality, watermark, text, deformed, ugly, "  
    "bad anatomy, out of frame, extra limbs, duplicate"  
)  
  
# —— 이미지 생성  
_____  
  
start = time.time()  
  
with torch.inference_mode(): # 그래디언트 계산 완전 비활성화  
    result = pipe(  
        prompt=prompt,  
        negative_prompt=negative_prompt,  
        width=512,  
        height=512,  
        num_inference_steps=50, # 역확산 스텝 수 (20~50 권장)  
        guidance_scale=7.5,    # CFG 스케일 (7~9 권장)  
        num_images_per_prompt=1, # 한 번에 생성할 이미지 수  
        generator=torch.Generator("cuda").manual_seed(42) # 재현성  
    )  
  
elapsed = time.time() - start  
print(f"생성 시간: {elapsed:.2f}초")  
  
# —— 이미지 저장  
_____  
  
os.makedirs("./outputs", exist_ok=True)  
  
image: Image.Image = result.images[0]  
output_path = f"./outputs/generated_{int(time.time())}.png"  
image.save(output_path, format="PNG")  
print(f"저장 완료: {output_path}")
```

```
# —— 이미지 정보 출력 ——
print(f"이미지 크기: {image.size} 모드: {image.mode}")
```

[코드 4-5] 첫 이미지 생성 및 PNG 파일 저장 전체 코드

```
# 복수 이미지 생성 및 그리드 저장 예시
from PIL import Image as PILImage
import math

def make_image_grid(images, rows, cols):
    """생성된 이미지들을 그리드로 합성"""
    w, h = images[0].size
    grid = PILImage.new("RGB", (cols * w, rows * h))
    for idx, img in enumerate(images):
        grid.paste(img, (idx % cols * w, idx // cols * h))
    return grid

# 4장 동시 생성 (다양한 시드로 비교)
images = []
for seed in [42, 137, 512, 999]:
    with torch.inference_mode():
        out = pipe(
            prompt=prompt,
            negative_prompt=negative_prompt,
            num_inference_steps=30,
            guidance_scale=7.5,
            generator=torch.Generator("cuda").manual_seed(seed)
        )
        images.append(out.images[0])

grid = make_image_grid(images, rows=2, cols=2)
grid.save("./outputs/grid_2x2.png")
print("그리드 이미지 저장 완료: ./outputs/grid_2x2.png")
```

[코드 4-6] 복수 이미지 생성 및 그리드 합성

💡 num_inference_steps를 50에서 20으로 줄이면 속도가 약 2.5배 빨라지면서도 품질 저하가 크지 않습니다. 빠른 테스트 시에는 20~30 스텝을, 최종 고품질 생성 시에는 50 스텝을 사용하세요.

4.2 프롬프트 입력 → 이미지 생성 API

(1) FastAPI로 이미지 생성 요청 API 구현

이미지 생성 파이프라인을 REST API로 감싸면 프론트엔드 앱, 모바일 앱, 다른 서비스에서 HTTP 요청만으로 AI 이미지를 생성할 수 있습니다. 이 절에서는 FastAPI 기반의 완전한 이미지 생성 API 서버를 구축합니다.

■ 프로젝트 디렉토리 구조

image_gen_api/

- |—— main.py — FastAPI 앱 진입점, 라우터 정의
- |—— schemas.py — Pydantic 요청/응답 스키마
- |—— pipeline_manager.py — 모델 로드 및 캐싱 관리
- |—— utils.py — 이미지 변환 유틸리티 함수
- |—— .env — 환경변수 (HF_TOKEN 등)
- |—— outputs/ — 생성된 이미지 임시 저장

```
# schemas.py - Pydantic 요청/응답 스키마
from pydantic import BaseModel, Field
from typing import Optional, Literal

class ImageGenRequest(BaseModel):
    prompt: str = Field(
        ...,
        min_length=3,
        max_length=500,
        description="이미지 생성에 사용할 영어 프롬프트",
        example="a serene Japanese garden at sunrise, ultra detailed, 4K"
    )
    negative_prompt: Optional[str] = Field(
        default="blurry, low quality, watermark, deformed",
```

```

        description="제외할 요소 목록"
    )
    width: int = Field(default=512, ge=256, le=1024, multiple_of=64)
    height: int = Field(default=512, ge=256, le=1024, multiple_of=64)
    num_inference_steps: int = Field(default=30, ge=10, le=100)
    guidance_scale: float = Field(default=7.5, ge=1.0, le=20.0)
    seed: Optional[int] = Field(default=None, description="재현성을 위한 랜덤 시드")
    response_format: Literal["base64", "file_url"] = Field(
        default="base64",
        description="응답 형식: base64 인코딩 문자열 or 파일 URL"
    )
)

class ImageGenResponse(BaseModel):
    success: bool
    image_data: Optional[str] = None # base64 인코딩 이미지
    file_url: Optional[str] = None # 저장된 이미지 URL
    generation_time_sec: float
    seed_used: int
    prompt: str
    model: str

```

[코드 4-7] Pydantic 요청/응답 스키마 (schemas.py)

```

# pipeline_manager.py - 모델 싱글턴 관리자
import torch
from diffusers import StableDiffusionPipeline
from threading import Lock
import logging

logger = logging.getLogger(__name__)

class PipelineManager:
    """모델을 싱글턴 패턴으로 관리, 스레드 안전 추론 지원"""
    _instance = None
    _lock = Lock()
    _inference_lock = Lock() # 동시 추론 방지 (VRAM 보호)

```

```

def __new__(cls):
    if cls._instance is None:
        with cls._lock:
            if cls._instance is None:
                cls._instance = super().__new__(cls)
                cls._instance._pipe = None
                cls._instance._model_id = None
            return cls._instance

def load(self, model_id: str, device: str = "cuda"):
    if self._pipe is not None and self._model_id == model_id:
        logger.info("파이프라인이 이미 로드되어 있습니다.")
        return

    logger.info(f"모델 로딩 시작: {model_id}")
    dtype = torch.float16 if device == "cuda" else torch.float32
    self._pipe = StableDiffusionPipeline.from_pretrained(
        model_id,
        torch_dtype=dtype,
        use_safetensors=True,
        safety_checker=None,
        requires_safety_checker=False
    ).to(device)
    self._pipe.set_progress_bar_config(disable=True)
    self._model_id = model_id
    logger.info("모델 로드 완료")

def generate(self, **kwargs) -> list:
    """스레드 안전 이미지 생성"""
    with self._inference_lock:
        with torch.inference_mode():
            result = self._pipe(**kwargs)
        return result.images

@property
def model_id(self): return self._model_id

pipeline_manager = PipelineManager()

```

[코드 4-8] 싱글턴 파이프라인 매니저 (pipeline_manager.py)

(2) 입력: 프롬프트 문자열, 출력: 이미지 파일 또는 base64 인코딩

이미지 반환 방식은 크게 두 가지입니다. base64 인코딩은 JSON 응답에 이미지 데이터를 직접 포함하여 추가 요청 없이 이미지를 전달하며, StreamingResponse는 이미지 파일을 직접 스트리밍하여 브라우저에서 바로 표시할 수 있습니다.

반환 방식	장점	단점	권장 사용 상황
base64 (JSON)	JSON에 포함, 단일 요청으로 완결	데이터 크기 33% 증가, 파싱 필요	API 통합, 프론트엔드 처리
StreamingResponse	직접 다운로드, 효율적 전송	별도 HTTP 요청 구조 필요	브라우저 직접 표시, 다운로드
파일 URL	대용량 이미지, CDN 연동 가능	스토리지 관리 필요	장기 보관, 대용량 이미지

```
# utils.py – 이미지 변환 유틸리티
import io, base64, os, time
from PIL import Image

def pil_to_base64(image: Image.Image, format: str = "PNG") -> str:
    """PIL Image를 base64 문자열로 변환"""
    buffer = io.BytesIO()
    image.save(buffer, format=format)
    buffer.seek(0)
    return base64.b64encode(buffer.read()).decode("utf-8")

def pil_to_bytes(image: Image.Image, format: str = "PNG") -> bytes:
    """PIL Image를 bytes로 변환"""
    buffer = io.BytesIO()
    image.save(buffer, format=format)
    buffer.seek(0)
    return buffer.read()

def save_image(image: Image.Image, output_dir: str = "./outputs") -> str:
    """이미지를 파일로 저장하고 경로 반환"""
```

```

os.makedirs(output_dir, exist_ok=True)
filename = f"img_{int(time.time()*1000)}.png"
path = os.path.join(output_dir, filename)
image.save(path)
return path

def base64_to_pil(b64_string: str) -> Image.Image:
    """base64 문자열을 PIL Image로 변환"""
    image_bytes = base64.b64decode(b64_string)
    return Image.open(io.BytesIO(image_bytes)).convert("RGB")

```

[코드 4-9] 이미지 변환 유틸리티 함수 모음 (utils.py)

```

# main.py – FastAPI 앱 메인
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException, BackgroundTasks
from fastapi.responses import StreamingResponse, FileResponse
from fastapi.staticfiles import StaticFiles
import torch, time, random, io, os

from schemas import ImageGenRequest, ImageGenResponse
from pipeline_manager import pipeline_manager
from utils import pil_to_base64, pil_to_bytes, save_image

MODEL_ID = "runwayml/stable-diffusion-v1-5"

@asynccontextmanager
async def lifespan(app: FastAPI):
    device = "cuda" if torch.cuda.is_available() else "cpu"
    pipeline_manager.load(MODEL_ID, device=device)
    yield
    torch.cuda.empty_cache()

app = FastAPI(
    title="AI 이미지 생성 API",
    description="Stable Diffusion 기반 텍스트 → 이미지 생성 서비스",
    version="1.0.0",

```

```
lifespan=lifespan
)

os.makedirs("./outputs", exist_ok=True)
app.mount("/outputs", StaticFiles(directory="outputs"), name="outputs")

@app.post("/api/v1/generate", response_model=ImageGenResponse)
async def generate_image(request: ImageGenRequest) -> ImageGenResponse:
    """텍스트 프롬프트로 이미지를 생성하고 base64 또는 URL로 반환합니다."""
    seed = request.seed if request.seed is not None else random.randint(0, 2**32 - 1)
    generator = torch.Generator("cuda" if torch.cuda.is_available() else "cpu")
    generator.manual_seed(seed)

    start = time.time()
    try:
        images = pipeline_manager.generate(
            prompt=request.prompt,
            negative_prompt=request.negative_prompt,
            width=request.width,
            height=request.height,
            num_inference_steps=request.num_inference_steps,
            guidance_scale=request.guidance_scale,
            generator=generator
        )
    except RuntimeError as e:
        if "out of memory" in str(e).lower():
            torch.cuda.empty_cache()
            raise HTTPException(status_code=507, detail="GPU 메모리 부족. 이미지 크기나 스텝을 줄여주세요.")
        raise HTTPException(status_code=500, detail=str(e))

    elapsed = time.time() - start
    image = images[0]

    if request.response_format == "base64":
        image_data = pil_to_base64(image)
        return ImageGenResponse(
            success=True, image_data=image_data,
```

```
        generation_time_sec=round(elapsed, 2),
        seed_used=seed, prompt=request.prompt,
        model=pipeline_manager.model_id
    )
else:
    path = save_image(image)
    filename = os.path.basename(path)
    return ImageGenResponse(
        success=True, file_url=f"/outputs/{filename}",
        generation_time_sec=round(elapsed, 2),
        seed_used=seed, prompt=request.prompt,
        model=pipeline_manager.model_id
    )
```

[코드 4-10] FastAPI 메인 앱 및 이미지 생성 엔드포인트 (main.py)

(3) Swagger UI에서 테스트 및 Streamlit 앱 병행

FastAPI 서버를 실행하고 Swagger UI에서 직접 이미지를 생성하는 방법을 안내합니다.

```
# 서버 실행 (터미널에서)
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# 접속 주소
# Swagger UI: http://localhost:8000/docs
# ReDoc:      http://localhost:8000/redoc

# — curl 테스트 —

curl -X POST "http://localhost:8000/api/v1/generate" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "a futuristic city at night, neon lights, rain reflection, 8K",
    "negative_prompt": "blurry, low quality, watermark",
    "width": 512, "height": 512,
    "num_inference_steps": 30,
    "guidance_scale": 7.5,
    "response_format": "base64"
  }' | python -c "
import sys, json, base64
data = json.load(sys.stdin)
img_bytes = base64.b64decode(data["image_data"])
open("result.png", "wb").write(img_bytes)
print(f'저장 완료: result.png ({data["generation_time_sec"]}초)')
```

[코드 4-11] 서버 실행 및 curl 테스트 (base64 응답 → PNG 저장)

Streamlit을 사용하면 웹 UI에서 바로 프롬프트를 입력하고 이미지를 확인할 수 있는 인터랙티브 앱을 빠르게 구축할 수 있습니다.

```
# streamlit_app.py
# 실행: streamlit run streamlit_app.py
```

```

import streamlit as st
import requests, base64, io
from PIL import Image

st.set_page_config(page_title="AI 이미지 생성기", page_icon="🐉", layout="wide")
st.title("🐉 AI 이미지 생성기 - Stable Diffusion")

API_URL = "http://localhost:8000/api/v1/generate"

col1, col2 = st.columns([1, 1])

with col1:
    st.subheader("🔮 생성 설정")
    prompt = st.text_area("프롬프트 (영어로 입력)", height=120,
        value="a majestic dragon flying over a medieval castle, sunset, epic, 4K")
    negative_prompt = st.text_area("부정 프롬프트", height=80,
        value="blurry, low quality, watermark, deformed")
    col_w, col_h = st.columns(2)
    width = col_w.selectbox("너비", [256, 512, 768], index=1)
    height = col_h.selectbox("높이", [256, 512, 768], index=1)
    steps = st.slider("추론 스텝 수", 10, 80, 30)
    cfg = st.slider("CFG SCALE (guidance_scale)", 1.0, 15.0, 7.5)
    seed = st.number_input("시드 (0=랜덤)", 0, 999999, 0)

    if st.button("🐉 이미지 생성", type="primary"):
        with st.spinner("이미지 생성 중..."):
            payload = {
                "prompt": prompt, "negative_prompt": negative_prompt,
                "width": width, "height": height,
                "num_inference_steps": steps, "guidance_scale": cfg,
                "seed": seed if seed > 0 else None,
                "response_format": "base64"
            }
            resp = requests.post(API_URL, json=payload, timeout=300)
            if resp.status_code == 200:

```

```
data = resp.json()
img_bytes = base64.b64decode(data["image_data"])
st.session_state["last_image"] = img_bytes
st.session_state["last_info"] = data
else:
    st.error(f"오류: {resp.json().get('detail', '알 수 없는 오류')}")

with col2:
    st.subheader("🖼️ 생성 결과")
    if "last_image" in st.session_state:
        img = Image.open(io.BytesIO(st.session_state["last_image"]))
        st.image(img, use_column_width=True)
        info = st.session_state["last_info"]
        st.caption(f"생성 시간: {info['generation_time_sec']}초 | 시드: {info['seed_used']}")
        st.download_button("📄 PNG 다운로드", st.session_state["last_image"],
                           file_name="generated.png", mime="image/png")
```

[코드 4-12] Streamlit 이미지 생성 앱 전체 코드 (streamlit_app.py)

(4) 이미지 생성 시간 최적화: torch.autocast / fp16

이미지 생성 속도와 메모리 효율을 개선하기 위한 주요 기법들을 소개합니다.

```
import torch
from diffusers import StableDiffusionPipeline

# — 방법 1: fp16 + torch.inference_mode (기본 권장) —
pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16, # FP16: 메모리 절반, 속도 향상
    use_safetensors=True,
    safety_checker=None,
    requires_safety_checker=False
).to("cuda")

with torch.inference_mode():
    image = pipe("a red fox in snow", num_inference_steps=30).images[0]

# — 방법 2: torch.autocast (구버전 호환, 동적 혼합 정밀도) —
with torch.autocast("cuda"):
    image = pipe("a red fox in snow", num_inference_steps=30).images[0]

# — 방법 3: xFormers 메모리 효율적 Attention —
# pip install xformers 필요
pipe.enable_xformers_memory_efficient_attention()
# VRAM 약 20~30% 절감, 속도 약 10~20% 향상

# — 방법 4: attention slicing (저사양 GPU용) —
# VRAM이 6GB 미만인 경우 사용. 속도는 느려지지만 VRAM 절감
pipe.enable_attention_slicing(1)

# — 방법 5: vae slicing (배치 처리 시 VRAM 절감) —
pipe.enable_vae_slicing()
```

```
# —— 방법 6: torch.compile (PyTorch 2.0+, 첫 실행 느림) ——
pipe.unet = torch.compile(pipe.unet, mode="reduce-overhead", fullgraph=True)
# 컴파일 후 반복 실행 시 20~50% 속도 향상

# —— 최적화 효과 측정 ——
import time
times = []
for _ in range(5):
    start = time.time()
    with torch.inference_mode():
        pipe("benchmark test", num_inference_steps=20)
    times.append(time.time() - start)

print(f"평균 생성 시간: {sum(times)/len(times):.2f}초")
print(f"VRAM 사용량: {torch.cuda.memory_allocated()/1e9:.2f} GB")
```

[코드 4-13] 다양한 속도/메모리 최적화 기법 비교

최적화 기법	속도 향상	VRAM 절감	조건
fp16 (torch.float16)	1.5~ 2x	50%	CUDA GPU 필수
xFormers	1.1~ 1.2x	20~ 30%	xformers 패키지 설치
attention slicing	0.7~ 0.9x (느려짐)	40~ 50%	VRAM 부족 환경용
vae slicing	비슷	배치 시 절감	다수 이미지 동시 생성 시
torch.compile	1.2~ 1.5x (반복 실행 시)	없음	PyTorch 2.0+, 초기 컴파일 비용
DDIM 20 스텝	2.5x (50→20)	없음	품질 소폭 저하 감수

4.3 ControlNet과 LoRA로 이미지 변환

(1) 이미지 구조 보존을 위한 ControlNet 개념 설명

ControlNet은 2023년 Stanford 연구팀이 발표한 기법으로, 기존 Stable Diffusion에 추가적인 조건 입력(Conditioning Input)을 제공하여 생성 이미지의 구조, 포즈, 깊이감 등을 정밀하게 제어할 수 있게 합니다. 텍스트 프롬프트만으로는 세밀한 구도와 구조를 원하는 대로 제어하기 어렵다는 SD의 한계를 극복하는 핵심 기술입니다.

ControlNet은 원본 UNet의 가중치를 동결(freeze)하고, 별도의 학습 가능한 복사본(Trainable Copy)을 통해 조건 신호를 주입합니다. 이를 통해 원본 모델의 이미지 품질을 유지하면서 구조 정보를 정확하게 반영할 수 있습니다.

ControlNet 유형	입력 조건	모델 ID	활용 예
Canny Edge	캐니 엣지 검출 이미지	llyasviel/sd-controlnet-canny	선화 → 채색, 스케치 변환
HED Edge	부드러운 엣지 맵	llyasviel/sd-controlnet-hed	윤곽 보존 스타일 변환
OpenPose	인체 관절 스켈레톤	llyasviel/sd-controlnet-openpose	자세 유지 캐릭터 생성
Depth Map	깊이 정보 맵	llyasviel/sd-controlnet-depth	3D 구도 보존 재구성
Normal Map	법선 벡터 맵	llyasviel/sd-controlnet-normal	3D 표면 재질 변환
Segmentation	의미적 분할 맵	llyasviel/sd-controlnet-seg	배경/객체 구조 보존
Scribble	손그림 낙서 맵	llyasviel/sd-controlnet-scribble	러프 스케치 → 완성 이미지
MLSD Line	직선 구조 검출	llyasviel/sd-controlnet-mlsd	건축물, 실내 구조 보존

ControlNet의 작동 방식을 단계별로 살펴보면 다음과 같습니다.

🔗 ControlNet 처리 흐름

- ① 조건 이미지 전처리: 원본 이미지 → 조건 맵 생성
예) 원본 사진 → Canny 엣지 검출 → 흑백 엣지 이미지
- ② ControlNet Encoder 처리: 조건 맵 → 중간 Feature Map 시퀀스 생성
(원본 UNet Encoder와 동일한 구조, 별도 학습 가중치)
- ③ Feature 주입: ControlNet의 출력이 원본 UNet의 각 디코더 블록에 더해짐
- ④ 통상적 역확산 진행: 텍스트 임베딩 + ControlNet 조건이 결합된 UNet으로
노이즈 제거 반복 → 최종 이미지 생성

(2) ControlNet Canny 예시 모델 실습

가장 널리 사용되는 ControlNet Canny를 이용하여 입력 이미지의 윤곽선 구조를 보존하면서 새로운 스타일의 이미지를 생성합니다.

```
import torch
import cv2
import numpy as np
from PIL import Image
from diffusers import StableDiffusionControlNetPipeline, ControlNetModel
from diffusers import UniPCMultistepScheduler

# —— Step 1: ControlNet 모델 로드 ——
controlnet = ControlNetModel.from_pretrained(
    "lllyasviel/sd-controlnet-canny",
    torch_dtype=torch.float16,
    use_safetensors=True
).to("cuda")
```

```
# —— Step 2: 파이프라인 조합 ——
pipe = StableDiffusionControlNetPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    controlnet=controlnet,
    torch_dtype=torch.float16,
    use_safetensors=True,
    safety_checker=None,
    requires_safety_checker=False
).to("cuda")

# UniPCMultistepScheduler: 고품질 + 빠른 생성 (20 스텝으로 충분)
pipe.scheduler = UniPCMultistepScheduler.from_config(pipe.scheduler.config)
pipe.enable_xformers_memory_efficient_attention()
```

[코드 4-14] ControlNet Canny 파이프라인 초기화

```
# —— Step 3: 입력 이미지에서 Canny 엣지 추출 ——
def extract_canny_edges(image_path: str, low_threshold=100, high_threshold=200) -> Image.Image:
    """이미지에서 Canny 엣지를 검출하여 ControlNet 입력 이미지 생성"""
    # 이미지 로드 및 리사이즈
    img = Image.open(image_path).convert("RGB").resize((512, 512))
    img_array = np.array(img)

    # BGR로 변환 후 Canny 엣지 검출
    img_bgr = cv2.cvtColor(img_array, cv2.COLOR_RGB2BGR)
    edges = cv2.Canny(img_bgr, low_threshold, high_threshold)

    # 3채널로 변환 (ControlNet 입력 형식)
    edges_rgb = cv2.cvtColor(edges, cv2.COLOR_GRAY2RGB)
    return Image.fromarray(edges_rgb)

# —— Step 4: ControlNet으로 이미지 생성 ——
def generate_with_canny(
    image_path: str,
    prompt: str,
    negative_prompt: str = "blurry, low quality, deformed",
    num_steps: int = 20,
    guidance_scale: float = 7.5,
```



```

controlnet_scale: float = 1.0, # ControlNet 조건 강도 (0.5~1.5)
seed: int = 42
) -> tuple[Image.Image, Image.Image]:
    """원본 이미지 엣지를 보존하며 새로운 스타일로 이미지 생성"""

    # Canny 엣지 맵 생성
    canny_image = extract_canny_edges(image_path)

    generator = torch.Generator("cuda").manual_seed(seed)

    with torch.inference_mode():
        result = pipe(
            prompt=prompt,
            negative_prompt=negative_prompt,
            image=canny_image,          # ControlNet 조건 입력
            num_inference_steps=num_steps,
            guidance_scale=guidance_scale,
            controlnet_conditioning_scale=controlnet_scale,
            generator=generator
        )

    return canny_image, result.images[0]

# — 실행 테스트 —
canny_map, generated = generate_with_canny(
    image_path="./sample_building.jpg",
    prompt="a futuristic cyberpunk building, neon lights, rain, ultra detailed, 8K",
    controlnet_scale=0.9
)

canny_map.save("./outputs/canny_map.png")
generated.save("./outputs/canny_result.png")
print("ControlNet Canny 결과 저장 완료")

```

[코드 4-15] Canny 엣지 추출 및 ControlNet 이미지 생성 전체 코드

💡 `controlnet_conditioning_scale` 값을 낮추면(0.3~0.5) 원본 구조에서 더 자유롭게 변형되고, 높이면(0.9~1.5) 구조를 더 강하게 보존합니다. 일반적으로 0.7~1.0이 균형잡힌 결과를 보여줍니다.

(3) Low-Rank Adaptation (LoRA)란 무엇인가?

LoRA(Low-Rank Adaptation)는 2021년 Microsoft Research에서 제안한 파라미터 효율적 파인튜닝(PEFT) 기법으로, 원본 모델의 대부분 가중치를 동결(freeze)하고 작은 행렬 두 개(저랭크 행렬 A와 B)만 학습합니다. Stable Diffusion에 적용하면 특정 아티스트 스타일, 캐릭터, 개념 등을 수십~수백 MB 크기의 소형 파일로 학습하고 배포할 수 있습니다.

LoRA의 수학적 원리

기존 파인튜닝: $W_{new} = W_{original} + \Delta W$ (ΔW 는 W 와 같은 크기)

LoRA: $W_{new} = W_{original} + \alpha \times (A \times B)$

- $W_{original}$: 기존 모델 가중치 (고정, 학습 안 함)
- A : 저랭크 행렬 ($d \times r$) - 학습 대상
- B : 저랭크 행렬 ($r \times k$) - 학습 대상
- r : 랭크 (보통 4~64. 작을수록 학습 파라미터 적음)
- α : 스케일링 인자

효과: $r=4$ 사용 시 학습 파라미터 수가 기존의 약 0.01~0.1% 수준으로 감소

→ 학습 시간 단축, 소형 파일 배포, 여러 LoRA 혼합 사용 가능

구분	일반 파인튜닝	LoRA 파인튜닝
학습 파라미터 수	수억 개 (전체)	수백만 개 (0.1% 수준)
파일 크기	2~7 GB	1~200 MB
학습 시간	수 시간 ~ 수 일	수십 분 ~ 수 시간
VRAM 요구	24 GB+	6~12 GB
여러 개 혼합	불가능	가중치로 혼합 가능
원본 모델 호환	재배포 필요	원본 모델 + LoRA 파일

(4) LoRA 가중치 로드 및 스타일 전환 실습

Hugging Face Hub 또는 civitai.com에서 다양한 LoRA 파일을 다운로드하여 기존 Stable Diffusion 모델에 적용할 수 있습니다. diffusers 라이브러리는 load_lora_weights() 메서드로 간편하게 LoRA를 로드합니다.

```
import torch
from diffusers import StableDiffusionPipeline

# — 기본 SD 파이프라인 로드 —
pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    use_safetensors=True,
    safety_checker=None,
    requires_safety_checker=False
).to("cuda")

# — LoRA 가중치 로드 방법 1: Hugging Face Hub에서 직접 로드 —
# 예: 애니메이션 스타일 LoRA
pipe.load_lora_weights(
    "sayakpaul/sd-model-finetuned-lora-t4",
    weight_name="pytorch_lora_weights.safetensors"
)

# — LoRA 가중치 로드 방법 2: 로컬 .safetensors 파일 로드 —
pipe.load_lora_weights(
    "./lora_weights/",          # 디렉토리 경로
    weight_name="anime_style.safetensors"
)

# — LoRA 스케일 조정 —
# cross_attention_kwargs={"scale": 0.7}로 LoRA 적용 강도 조절
with torch.inference_mode():
    image_lora = pipe(
        "a beautiful anime girl in a garden, cherry blossoms, soft lighting",
        negative_prompt="realistic, photo, 3D render, deformed",
        num_inference_steps=30,
```

```

        guidance_scale=7.5,
        cross_attention_kwargs={"scale": 0.8}, # LoRA 강도: 0~1
    ).images[0]

image_lora.save("./outputs/lora_result.png")

# ——— 여러 LoRA 혼합 사용 ———
# (diffusers 0.26+에서 지원)
pipe.load_lora_weights("./lora_weights/", weight_name="style_A.safetensors",
                        adapter_name="style_A")
pipe.load_lora_weights("./lora_weights/", weight_name="character_B.safetensors",
                        adapter_name="character_B")

# 두 LoRA를 6:4 비율로 혼합
pipe.set_adapters(["style_A", "character_B"], adapter_weights=[0.6, 0.4])

# 혼합 LoRA로 생성
with torch.inference_mode():
    image_mixed = pipe(
        "a warrior princess in a fantasy world, dramatic lighting",
        num_inference_steps=30
    ).images[0]
image_mixed.save("./outputs/mixed_lora.png")

# ——— LoRA 비활성화 (원본 모델로 복원) ———
pipe.unload_lora_weights()

# ——— LoRA를 모델에 영구 병합 (배포 시 권장) ———
# 병합 후에는 원본 SD 파이프라인처럼 사용 가능 (LoRA 파일 불필요)
pipe.fuse_lora(lora_scale=1.0)
pipe.save_pretrained("./merged_model/")

```

[코드 4-16] LoRA 로드, 스케일 조정, 혼합, 병합 전체 예시

(5) 업로드된 이미지 → 변환된 이미지 반환 API 구현

사용자가 이미지를 업로드하면 ControlNet 또는 img2img 파이프라인으로 변환하여 반환하는 REST API를 구현합니다. FastAPI의 multipart/form-data 처리를 통해 이미지 파일을 수신합니다.

```
# image_transform_router.py - 이미지 변환 라우터
import io, torch
from fastapi import APIRouter, UploadFile, File, Form, HTTPException
from fastapi.responses import StreamingResponse
from PIL import Image
from diffusers import (
    StableDiffusionImg2ImgPipeline,
    StableDiffusionControlNetPipeline,
    ControlNetModel,
    UniPCMultistepScheduler
)
from diffusers.utils import load_image
import numpy as np, cv2

router = APIRouter(prefix="/api/v1/transform", tags=["Image Transform"])

# —— 파이프라인 초기화 (앱 시작 시 한 번만 실행) ——
_img2img_pipe = None
_controlnet_pipe = None

def get_img2img_pipe():
    global _img2img_pipe
    if _img2img_pipe is None:
        _img2img_pipe = StableDiffusionImg2ImgPipeline.from_pretrained(
            "runwayml/stable-diffusion-v1-5",
            torch_dtype=torch.float16,
            use_safetensors=True,
            safety_checker=None,
            requires_safety_checker=False
        ).to("cuda")
    return _img2img_pipe
```

```

def get_controlnet_pipe():
    global _controlnet_pipe
    if _controlnet_pipe is None:
        cn = ControlNetModel.from_pretrained(
            "lllyasviel/sd-controlnet-canny",
            torch_dtype=torch.float16, use_safetensors=True
        ).to("cuda")
        _controlnet_pipe = StableDiffusionControlNetPipeline.from_pretrained(
            "runwayml/stable-diffusion-v1-5",
            controlnet=cn,
            torch_dtype=torch.float16,
            use_safetensors=True,
            safety_checker=None,
            requires_safety_checker=False
        ).to("cuda")
        _controlnet_pipe.scheduler = UniPCMultistepScheduler.from_config(
            _controlnet_pipe.scheduler.config)
    return _controlnet_pipe

```

[코드 4-17] 이미지 변환 라우터 및 파이프라인 초기화

```

# — img2img 엔드포인트 —————
@router.post("/img2img", summary="이미지 스타일 변환 (img2img)")
async def img2img_transform(
    file: UploadFile = File(..., description="변환할 원본 이미지 파일 (JPG/PNG)"),
    prompt: str = Form(..., description="변환 스타일 프롬프트"),
    negative_prompt: str = Form(default="blurry, low quality"),
    strength: float = Form(default=0.75, ge=0.1, le=1.0,
                           description="변형 강도. 1.0=완전 변형, 0.1=원본 유지"),
    num_steps: int = Form(default=30, ge=10, le=80),
    guidance_scale: float = Form(default=7.5)
):
    # 이미지 읽기 및 전처리
    content = await file.read()
    if not file.content_type.startswith("image/"):
        raise HTTPException(400, "이미지 파일만 업로드 가능합니다.")

```

```

init_image = Image.open(io.BytesIO(content)).convert("RGB").resize((512, 512))
pipe = get_img2img_pipe()

with torch.inference_mode():
    result = pipe(
        prompt=prompt,
        negative_prompt=negative_prompt,
        image=init_image,
        strength=strength,      # 변형 강도
        num_inference_steps=num_steps,
        guidance_scale=guidance_scale,
    )

output_image = result.images[0]
buf = io.BytesIO()
output_image.save(buf, format="PNG")
buf.seek(0)
return StreamingResponse(buf, media_type="image/png",
    headers={"Content-Disposition": "inline; filename=transformed.png"})

# — ControlNet 변환 엔드포인트 —
@router.post("/controlnet-canny", summary="ControlNet Canny 구조 보존 변환")
async def controlnet_canny_transform(
    file: UploadFile = File(...),
    prompt: str = Form(...),
    negative_prompt: str = Form(default="blurry, deformed, low quality"),
    num_steps: int = Form(default=20),
    controlnet_scale: float = Form(default=1.0, ge=0.1, le=2.0),
):
    content = await file.read()
    orig_image = Image.open(io.BytesIO(content)).convert("RGB").resize((512, 512))

    # Canny 엣지 추출
    arr = np.array(orig_image)
    bgr = cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)
    edges = cv2.Canny(bgr, 100, 200)

```



```

canny_image = Image.fromarray(cv2.cvtColor(edges, cv2.COLOR_GRAY2RGB))

pipe = get_controlnet_pipe()
with torch.inference_mode():
    result = pipe(
        prompt=prompt,
        negative_prompt=negative_prompt,
        image=canny_image,
        num_inference_steps=num_steps,
        controlnet_conditioning_scale=controlnet_scale,
    )

buf = io.BytesIO()
result.images[0].save(buf, format="PNG")
buf.seek(0)
return StreamingResponse(buf, media_type="image/png")

```

[코드 4-18] img2img 및 ControlNet Canny 변환 엔드포인트 전체 코드

★ strength 파라미터는 img2img 파이프라인에서 원본 이미지를 얼마나 변형할지 결정합니다. 0.3~0.5이면 원본 이미지와 유사하면서 스타일만 변경되고, 0.7~0.9이면 구도는 비슷하지만 크게 변형됩니다.

4.4 서버에서 모델 최적화하기

(1) GPU 메모리 최적화

Stable Diffusion 모델은 기본적으로 상당한 GPU 메모리(VRAM)를 필요로 합니다. SD v1.5를 fp32로 로드하면 약 8GB, fp16으로는 약 4GB의 VRAM을 사용합니다. VRAM이 제한된 환경에서는 다양한 최적화 기법을 조합하여 메모리 사용량을 줄여야 합니다.

VRAM 용량	가능한 모델 / 설정	권장 최적화 조합
4 GB	SD v1.5 (fp16, 512px)	fp16 + attention_slicing + vae_slicing
6 GB	SD v1.5 (fp16, 512px), SD v2 (fp16, 512px)	fp16 + xFormers + attention_slicing
8 GB	SD v1.5/v2 (fp16, 768px), ControlNet (512px)	fp16 + xFormers
12 GB	SD v1.5/v2 (fp16, 768px), ControlNet (768px)	fp16 + xFormers (선택)
16 GB+	SDXL (fp16, 1024px), 배치 생성 가능	fp16 (필수), 나머지 선택적
24 GB+	SDXL + ControlNet, 대형 배치	최적화 없이도 쾌적한 실행

```
import torch
from diffusers import StableDiffusionPipeline

# —— 현재 VRAM 사용량 모니터링 함수 ——
def print_gpu_memory(label: str = ""):
    if torch.cuda.is_available():
        allocated = torch.cuda.memory_allocated() / 1e9
        reserved = torch.cuda.memory_reserved() / 1e9
        total = torch.cuda.get_device_properties(0).total_memory / 1e9
        print(f"[{label}] VRAM: 사용={allocated:.2f}GB / 예약={reserved:.2f}GB / 전체={total:.2f}GB")

print_gpu_memory("모델 로드 전")
```

```

# —— 기본 메모리 최적화 파이프라인 초기화 ——
pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    use_safetensors=True,
    safety_checker=None,
    requires_safety_checker=False
).to("cuda")

print_gpu_memory("기본 로드 후")

# —— xFormers 활성화 (VRAM 20~30% 절감) ——
try:
    pipe.enable_xformers_memory_efficient_attention()
    print("xFormers 활성화 성공")
except Exception:
    print("xFormers 미설치 - attention_slicing으로 대체")
    pipe.enable_attention_slicing(1)

print_gpu_memory("최적화 적용 후")

# —— 사용하지 않는 구성 요소 언로드 ——
# 텍스트 인코더는 생성 전에 한 번만 필요 → 사용 후 오프로드 가능
def generate_and_offload(pipe, prompt, **kwargs):
    """생성 후 텍스트 인코더를 CPU로 이동하여 VRAM 확보"""
    image = pipe(prompt, **kwargs).images[0]
    # 텍스트 인코더를 CPU로 이동
    pipe.text_encoder.to("cpu")
    torch.cuda.empty_cache()
    print_gpu_memory("생성 후 오프로드")
    # 다음 생성 전에 GPU로 복귀 필요
    pipe.text_encoder.to("cuda")
    return image

```

[코드 4-19] VRAM 모니터링 및 기본 메모리 최적화

(2) enable_model_cpu_offload()로 메모리 절약

enable_model_cpu_offload()는 diffusers가 제공하는 강력한 메모리 최적화 기법입니다. 모델의 각 구성 요소(Text Encoder, UNet, VAE)를 필요할 때만 GPU에 올리고, 사용 후에는 자동으로 CPU로 이동하여 VRAM 사용량을 최소화합니다.

```
import torch

from diffusers import StableDiffusionPipeline, StableDiffusionXLPipeline


# —— CPU Offload 방법 1: enable_model_cpu_offload ——
# 모델 구성 요소를 자동으로 CPU ↔ GPU 이동
# accelerate 패키지 필요: pip install accelerate
pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    use_safetensors=True,
    safety_checker=None,
    requires_safety_checker=False
) # ← .to("cuda") 하지 않음!


# CPU Offload 활성화 (4GB GPU에서도 SD 실행 가능)
pipe.enable_model_cpu_offload()


# 이후 사용법은 동일
with torch.inference_mode():
    image = pipe("a sunset over the ocean, impressionist painting style").images[0]
image.save("./outputs/cpu_offload_test.png")


# —— CPU Offload 방법 2: enable_sequential_cpu_offload ——
# 더 세밀한 레이어 단위 오프로드. VRAM 최소화하지만 느림
pipe2 = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    use_safetensors=True,
    safety_checker=None,
```

```

requires_safety_checker=False
)
pipe2.enable_sequential_cpu_offload() # 모든 레이어를 순차 오프로드

# ----- SDXL에서의 CPU Offload -----
# SDXL은 10GB+ VRAM 필요 → CPU Offload로 8GB GPU에서 실행 가능
pipe_xl = StableDiffusionXLPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.float16,
    use_safetensors=True,
    variant="fp16"
)
pipe_xl.enable_model_cpu_offload()

with torch.inference_mode():
    image_xl = pipe_xl(
        "a breathtaking landscape, masterpiece quality, 8K",
        num_inference_steps=25,
    ).images[0]
image_xl.save("./outputs/sdxl_cpu_offload.png")

```

[코드 4-20] enable_model_cpu_offload() 및 sequential_cpu_offload 사용법

Offload 방법	VRAM 사용량	속도	설명
.to("cuda") (기본)	전체 (~4GB fp16)	가장 빠름	모든 구성 요소가 GPU에 상주
enable_model_cpu_offload()	최소 ~2.5GB	약간 느림	사용하지 않는 구성 요소는 CPU로
enable_sequential_cpu_offload()	최소 ~1.5GB	느림	레이어 단위 순차 오프로드

(3) 실시간 생성 속도 개선 전략

서비스 환경에서 사용자 대기 시간을 최소화하기 위한 다양한 전략을 소개합니다.

```
import torch
from diffusers import StableDiffusionPipeline, LCMScheduler

# —— 전략 1: LCM (Latent Consistency Model) 스케줄러 ——
# 4~8 스텝으로 고품질 이미지 생성 가능! 기존 30~50 스텝 대비 5~10배 빠름
# pip install diffusers[torch] --upgrade 필요

pipe_lcm = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    use_safetensors=True,
    safety_checker=None,
    requires_safety_checker=False
).to("cuda")

# LCM-LoRA 로드 (LCM 속도를 일반 SD 모델에 적용)
pipe_lcm.load_lora_weights("latent-consistency/lcm-lora-sdv1-5")
pipe_lcm.scheduler = LCMScheduler.from_config(pipe_lcm.scheduler.config)

# 4 스텝으로 빠른 생성
import time
start = time.time()
with torch.inference_mode():
    image_lcm = pipe_lcm(
        "a beautiful forest waterfall, golden light, photorealistic",
        num_inference_steps=4,    # 기존 30~50 스텝 대신 4 스텝!
        guidance_scale=1.0,      # LCM은 낮은 CFG 권장
        cross_attention_kwargs={"scale": 1.0}
    ).images[0]
print(f"LCM 4-step 생성 시간: {time.time()-start:.2f}초")
image_lcm.save("./outputs/lcm_4step.png")

# —— 전략 2: Tiny AutoEncoder (TAESD) ——
# VAE 디코딩 단계를 초경량 TAESD로 대체 → 최종 디코딩 3~5배 빠름
# 품질 소폭 저하 있으나 실시간 프리뷰에 적합
```

```

from diffusers import AutoencoderTiny

pipe.vae = AutoencoderTiny.from_pretrained(
    "madebyollin/taesd",
    torch_dtype=torch.float16
).to("cuda")

# —— 전략 3: 워밍업 실행으로 첫 응답 지연 제거 ——
# torch.compile 사용 시 첫 실행이 매우 느림 → 워밍업으로 해결
print("모델 워밍업 중...")
with torch.inference_mode():
    _ = pipe("warmup", num_inference_steps=1) # 1 스텝으로 빠른 워밍업
torch.cuda.empty_cache()
print("워밍업 완료. 이제 빠른 응답 가능.")

# —— 전략 4: 배치 생성 파이프라인 (처리량 최대화) ——
def batch_generate(prompts: list, **kwargs) -> list:
    """여러 프롬프트를 한 번의 forward pass로 처리"""
    with torch.inference_mode():
        results = pipe(prompts, **kwargs) # prompts = 리스트로 전달
    return results.images

prompts = [
    "a red sports car on a mountain road",
    "a blue sailboat on a calm lake",
    "a golden wheat field at dusk"
]

start = time.time()
images_batch = batch_generate(prompts, num_inference_steps=20)
elapsed = time.time() - start
print(f"배치(3장) 생성 시간: {elapsed:.2f}초 (장당 {elapsed/3:.2f}초)")

```

[코드 4-21] LCM 스케줄러, TAESD, 배치 생성 등 속도 최적화 전략

속도 개선 전략	속도 향상	품질 영향	권장 사용 상황
LCM -LoRA (4 스텝)	5~ 10x	약간 저하	실시간 미리보기, 저지연 서비스
DDIM 20 스텝 (기본 50→20)	2.5x	미미한 저하	일반 API 서비스
torch.compile (반복 실행)	1.2~ 1.5x	없음	고트래픽 프로덕션
배치 처리 (3~ 4장)	2~ 3x (처리량)	없음	대량 생성 파이프라인
xFormers	1.1~ 1.2x	없음	모든 GPU 환경
TAESD VAE	1.2~ 1.5x	소폭 저하	실시간 프리뷰

(4) 로컬 서버 / Colab / Hugging Face Spaces 환경에서의 차이점

Stable Diffusion 서비스를 배포하는 환경은 크게 로컬 서버, Google Colab, Hugging Face Spaces의 세 가지로 나눌 수 있습니다. 각 환경의 특성을 이해하고 적합한 환경을 선택해야 합니다.

환경	장점	단점	적합한 상황
로컬 서버 (자체 GPU)	완전한 제어권, 데이터 보안, 지속 실행, 빠른 응답	초기 하드웨어 비용 높음, 유지관리 필요	프로덕션 서비스, 민감 데이터 처리
Google Colab (무료)	무료 GPU(T4/A100), 빠른 시작, 설치 불필요	세션 제한(12h), 지속 서비스 불가, 속도 불안정	학습/실습, 빠른 프로토타이핑
Google Colab Pro	A100 GPU, 더 긴 세션 유지, 빠른 속도	유료, 지속 서비스 여전히 어려움	집중 개발, 대용량 실험
Hugging Face Spaces	무료 CPU/GPU, 배포 간편, 공유 용이, 자동 HTTPS	무료 GPU 큐 대기 있음, 프로 플랜 필요	AI 데모, 포트폴리오, 소규모 서비스
AWS/GCP/Azure (클라우드)	유연한 스케일링, 관리형 인프라, SLA 보장	비용 높음, 설정 복잡	엔터프라이즈, 대규모 서비스

Hugging Face Spaces에 Gradio 앱으로 배포하는 방법을 소개합니다. Spaces는 GitHub과 유사한 방식으로 소스코드를 업로드하면 자동으로 배포됩니다.

```
# app.py - Hugging Face Spaces용 Gradio 앱
# 이 파일을 HF Space 저장소에 업로드하면 자동으로 배포됩니다

import gradio as gr
import torch
from diffusers import StableDiffusionPipeline

# —— 모델 로드

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
```

```

    use_safetensors=True,
    safety_checker=None,
    requires_safety_checker=False
)

# Spaces GPU 환경에 맞게 자동 설정
if torch.cuda.is_available():
    pipe = pipe.to("cuda")
    pipe.enable_xformers_memory_efficient_attention()
else:
    pipe.enable_model_cpu_offload()

# — 생성 함수
def generate(prompt, negative_prompt, steps, guidance, seed):
    generator = torch.Generator().manual_seed(int(seed))
    with torch.inference_mode():
        image = pipe(
            prompt=prompt,
            negative_prompt=negative_prompt,
            num_inference_steps=int(steps),
            guidance_scale=guidance,
            generator=generator
        ).images[0]
    return image

# — Gradio UI 정의
with gr.Blocks(title="AI 이미지 생성기") as demo:
    gr.Markdown("## 🌀 Stable Diffusion 이미지 생성기")
    with gr.Row():
        with gr.Column():
            prompt_input = gr.Textbox(label="프롬프트", lines=3,
                                       placeholder="a beautiful landscape, 4K, photorealistic")
            neg_input = gr.Textbox(label="부정 프롬프트", lines=2,
                                   value="blurry, low quality, watermark")
            steps_slider = gr.Slider(10, 50, value=25, step=5, label="스텝 수")
            cfg_slider = gr.Slider(1.0, 15.0, value=7.5, label="CFG Scale")

```

```
seed_number = gr.Number(value=42, label="시드")
btn = gr.Button("🖼️ 이미지 생성", variant="primary")
with gr.Column():
    output_image = gr.Image(label="생성 결과", type="pil")
btn.click(generate,
          inputs=[prompt_input, neg_input, steps_slider, cfg_slider, seed_number],
          outputs=output_image)

demo.launch()

# requirements.txt (Space 저장소 루트에 함께 업로드)
# diffusers==0.27.2
# transformers==4.40.1
# accelerate==0.29.3
# torch
# gradio
# xformers
```

[코드 4-22] Hugging Face Spaces용 Gradio 앱 전체 코드

4장 마무리

이 장에서는 Hugging Face diffusers 라이브러리를 활용한 Stable Diffusion 이미지 생성 서비스를 처음부터 끝까지 구축했습니다. 4.1절에서 SD의 작동 원리와 첫 이미지 생성을 시작으로, 4.2절에서 FastAPI 기반 REST API 서버를, 4.3절에서 ControlNet과 LoRA를 활용한 고급 이미지 변환 기법을, 4.4절에서는 프로덕션 환경을 위한 메모리 최적화와 속도 향상 전략을 학습했습니다.

다음 표는 이 장에서 다룬 핵심 기법들과 실무 적용 포인트를 정리한 것입니다.

섹션	핵심 기술	실무 적용
4.1 SD 기초	StableDiffusionPipeline, 토큰 발급, 첫 이미지 생성	프로토타이핑, 이미지 생성 기반 구축
4.2 이미지 생성 API	FastAPI + lifespan + base64/StreamingResponse	프론트-백 연동, AI 이미지 서비스 배포
4.3 ControlNet	Canny/Pose/Depth 조건 이미지, img2img	구도 보존 변환, 스케치→완성 이미지
4.3 LoRA	load_lora_weights, 혼합, 병합	스타일 전환, 경량 파인튜닝 배포
4.4 최적화	fp16, xFormers, CPU Offload, LCM, 배치	저지연 API, 저사양 GPU 배포

🔗 다음 장 미리보기 - 5장: 멀티모달 서비스 구축 (비전 + 언어)

5장에서는 이미지와 텍스트를 함께 처리하는 멀티모달 AI 서비스를 구축합니다.

- BLIP-2 / LLaVA: 이미지 설명 자동 생성 (Image Captioning)
- CLIP: 이미지-텍스트 유사도 검색 서비스
- OpenAI GPT-4V API 연동 및 Hugging Face 로컬 대안 비교
- 이미지 기반 질의응답(VQA) 웹 서비스 구현

연습 문제

6. StableDiffusionPipeline을 로드하고 다음 프롬프트들로 각각 이미지를 생성한 뒤, num_inference_steps를 20, 35, 50으로 달리하여 생성된 이미지의 품질 차이를 비교하고 결과를 정리하세요. 프롬프트: "a majestic eagle soaring above snowy mountains, golden light, 4K"
7. 4.2절의 FastAPI 이미지 생성 API 서버를 로컬에 구동하고, Swagger UI(/docs)에서 직접 프롬프트를 입력하여 이미지를 생성해 보세요. 생성된 이미지를 base64 형식으로 받아 PNG 파일로 저장하는 Python 클라이언트 스크립트를 작성하세요.
8. ControlNet Canny 파이프라인을 사용하여 임의의 건물 또는 인물 사진을 입력으로, 다음의 두 프롬프트로 각각 이미지를 생성하고 비교하세요. (A) "a futuristic cyberpunk building, neon lights" (B) "a traditional Japanese architecture, cherry blossoms". controlnet_conditioning_scale을 0.5와 1.0으로 각각 적용하여 결과 차이도 관찰하세요.
9. enable_model_cpu_offload()를 적용한 파이프라인과 적용하지 않은 파이프라인의 (A) VRAM 사용량과 (B) 이미지 생성 시간을 측정하여 비교 분석하는 코드를 작성하세요. GPU 환경이 없다면 torch.cuda.memory_allocated() 부분을 제외하고 시간만 비교해도 됩니다.
10. (심화) Hugging Face Spaces에 무료 계정으로 Space를 만들고, 4.4절의 Gradio 앱 코드(app.py)를 업로드하여 실제로 배포해 보세요. 배포된 Space URL을 공유할 수 있게 구성하고, 배포 과정에서 발생한 오류와 해결 방법을 정리하세요.

— 4장 끝 —